ITA0612-Machine Learning

CO2-ASSESSMENT-1

Divyesh S P (192424147)

Question-1:

A Genetic Algorithm is applied to maximize the function f(x) = x^2 using an initial population consisting of the values 2, 4, 6, and 8. Determine the fitness value for each individual in the population and select the best candidates based on their fitness scores. Perform a crossover operation by exchanging the least significant bits of the selected individuals to generate new offspring. Additionally, explain the role of mutation in improving the diversity and effectiveness of the solution in Genetic Algorithms.

Code:

```
# Genetic Algorithm Example

# Initial population
population = [2, 4, 6, 8]

# Fitness function
def fitness(x):
    return x**2

# Calculate fitness
fitness_values = [fitness(x) for x in population]

print("Population:", population)
print("Fitness Values:", fitness_values)

# Select best two individuals
sorted_population = sorted(population, key=fitness, reverse=True)
parents = sorted_population[:2]
```

```python
print("\nSelected Parents:", parents)

# Convert to 4-bit binary
p1 = format(parents[0], '04b')
p2 = format(parents[1], '04b')

print("Parent 1:", p1)
print("Parent 2:", p2)

# Exchange Least Significant Bit (LSB)
child1 = p1[:-1] + p2[-1]
child2 = p2[:-1] + p1[-1]

print("\nAfter Crossover")
print("Child 1:", child1, "=", int(child1, 2))
print("Child 2:", child2, "=", int(child2, 2))

# Mutation (Flip last bit of Child 1)
mutation = list(child1)
mutation[-1] = '1' if mutation[-1] == '0' else '0'
mutation = ''.join(mutation)

print("\nAfter Mutation")
print("Mutated Child:", mutation, "=", int(mutation, 2))
```

Output:

```
Population: [2, 4, 6, 8]
Fitness Values: [4, 16, 36, 64]

Selected Parents: [8, 6]
Parent 1: 1000
Parent 2: 0110

After Crossover
Child 1: 1000 = 8
Child 2: 0110 = 6

After Mutation
Mutated Child: 1001 = 9
```

Role of Mutation

Mutation randomly changes one or more bits in a chromosome.

Example:

1000 → 1001 (8 → 9)

Importance of Mutation

- Maintains genetic diversity.
- Prevents premature convergence.
- Helps escape local optima.
- Improves the chances of finding the global optimum.

Question-2:

A perceptron model is used to classify whether a customer will purchase a product based on two features: advertisement exposure and income level. The weights are 0.5 and 0.3, and the bias is –0.4. For a customer with advertisement exposure = 2 and income level = 1, compute the net input to the perceptron and apply the step activation function. Determine whether the customer makes a purchase or not.

Code:

```
# Perceptron Example

# Inputs
x1 = 2
x2 = 1

# Weights
w1 = 0.5
w2 = 0.3

# Bias
b = -0.4

# Net input
net_input = w1*x1 + w2*x2 + b

print("Net Input =", net_input)

# Step Activation Function
if net_input >= 0:
    output = 1
else:
    output = 0

print("Perceptron Output =", output)

if output == 1:
    print("Customer will purchase the product.")
else:
    print("Customer will not purchase the product.")
```

Output:

```
Net Input = 0.9
Perceptron Output = 1
Customer will purchase the product.
```